Department of Artificial Intelligence and Machine Learning
B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)
EXPERIMENT NO. 6

AIM:
To write operational schemas for an e-commerce catalog tracking system connecting Next.js with MongoDB through a Mongoose modeling structure.

SOFTWARE REQUIRED:
- **Operating System:** Linux / macOS / Windows 10/11
- **Database Engine:** MongoDB (Local instance, Docker Container, or MongoDB Atlas)
- **Node.js runtime environment:** (v18.17.0 or higher)
- **Package Manager:** npm or pnpm
- **Code Editor:** Visual Studio Code

THEORY:
- **Document-Oriented Data Modeling with MongoDB:** MongoDB stores data as flexible, JSON-like BSON (Binary JSON) documents. Unlike rigid relational database management systems (RDBMS), MongoDB allows semi-structured schemas, polymorphic documents, and nested sub-documents, which are ideal for dynamic e-commerce product catalogs where attributes vary across categories.
- **Schema Abstraction with Mongoose:** Mongoose provides an Object Data Modeling (ODM) layer on top of the MongoDB native driver. It enforces strict application-level schema definitions, automatic type casting, field validation rules, default values, pre/post middleware lifecycle hooks, and virtual population across collections using Schema.Types.ObjectId.
- **Database Connection Caching in Next.js Serverless Environment:** Next.js Server Components and Server Actions execute in a stateless, ephemeral serverless/HMR environment. Calling mongoose.connect() inside standard route handlers or Server Actions without a connection caching mechanism creates a brand-new connection on every hot-reload or request, quickly exhausting the MongoDB connection pool. Caching the active connection promise on Node.js globalThis guarantees that server functions reuse existing connection handles across hot-module reloads.
- **Embedded Documents vs. Document Referencing:**
    - **Embedded (Denormalized) Sub-documents:** Used for attributes that are always retrieved together with the product, such as metadata, technical specifications, and inventory tracking dimensions.
    - **Referenced (Normalized) Documents:** Used for independent domain entities like Category or Vendor, mapped via ref pointers and populated using Mongoose .populate().

STEPS / ALGORITHM:

Step 1: Install Mongoose Dependencies

Install Mongoose in your Next.js project:

```
npm install mongoose
```

Configure your MongoDB connection string in .env:

```
MONGODB_URI="mongodb://localhost:27017/nextjs_ecommerce_catal
og"
```

Step 2: Establish MongoDB Global Connection Cache (app/lib/mongodb.ts**)**

Create a persistent connection utility that caches the MongoDB connection promise globally to prevent connection leaks during Next.js hot-reloads.

```
import mongoose from 'mongoose';

const MONGODB_URI = process.env.MONGODB_URI;

if (!MONGODB_URI) {
  throw new Error('Please define the MONGODB_URI environment
variable inside .env');
}

interface MongooseCache {
  conn: typeof mongoose | null;
  promise: Promise<typeof mongoose> | null;
}

declare global {
  var mongooseCache: MongooseCache | undefined;
}

let cached = global.mongooseCache;

if (!cached) {
  cached = global.mongooseCache = { conn: null, promise: null
};
}

export async function connectToDatabase(): Promise<typeof
mongoose> {
  if (cached!.conn) {
```

```
    return cached!.conn;
  }

  if (!cached!.promise) {
    const opts = {
      bufferCommands: false,
    };

    cached!.promise = mongoose.connect(MONGODB_URI as string,
opts).then((mongooseInstance) => {
      return mongooseInstance;
    });
  }

  try {
    cached!.conn = await cached!.promise;
  } catch (e) {
    cached!.promise = null;
    throw e;
  }

  return cached!.conn;
}
```

Step 3: Define Operational Mongoose Schemas (app/models/Product.ts**)**
Construct an e-commerce catalog schema incorporating embedded specification structures, enumeration constraints, validation rules, and inventory tracking.

```
import mongoose, { Schema, Document, Model } from 'mongoose';

// Interface for embedded inventory sub-document
export interface IInventory {
  sku: string;
  stock: number;
  warehouseLocation: string;
}

// Interface for the Product Document
export interface IProduct extends Document {
  title: string;
  slug: string;
  price: number;
```

```
  category: string;
  tags: string[];
  inventory: IInventory;
  inStock: boolean;
  createdAt: Date;
  updatedAt: Date;
}

// Embedded Inventory Schema
const InventorySchema = new Schema<IInventory>(
  {
    sku: {
      type: String,
      required: [true, 'SKU identifier is mandatory'],
      uppercase: true,
      trim: true,
    },
    stock: {
      type: Number,
      required: [true, 'Stock count is required'],
      min: [0, 'Stock cannot be negative'],
      default: 0,
    },
    warehouseLocation: {
      type: String,
      required: [true, 'Warehouse location must be
specified'],
      trim: true,
    },
  },
  { _id: false }
);

// Main Product Catalog Schema
const ProductSchema = new Schema<IProduct>(
  {
    title: {
      type: String,
      required: [true, 'Product title is required'],
      trim: true,
      maxlength: [100, 'Title cannot exceed 100 characters'],
```

```
    },
    slug: {
      type: String,
      required: [true, 'Product slug is required'],
      unique: true,
      lowercase: true,
      trim: true,
    },
    price: {
      type: Number,
      required: [true, 'Price must be specified'],
      min: [0, 'Price cannot be negative'],
    },
    category: {
      type: String,
      required: [true, 'Category is required'],
      enum: ['Electronics', 'Footwear', 'Apparel',
'Accessories', 'Home'],
    },
    tags: {
      type: [String],
      default: [],
    },
    inventory: {
      type: InventorySchema,
      required: true,
    },
    inStock: {
      type: Boolean,
      default: true,
    },
  },
  {
    timestamps: true,
  }
);

// Pre-save hook: Automatically evaluate stock status based
on inventory levels
ProductSchema.pre<IProduct>('save', function (next) {
  this.inStock = this.inventory.stock > 0;
```

```
  next();
});

// Prevent model recompilation errors in Next.js development
HMR
const Product: Model<IProduct> =
  mongoose.models.Product ||
mongoose.model<IProduct>('Product', ProductSchema);

export default Product;
```

Step 4: Implement Catalog Server Actions (app/actions/catalogActions.ts**)**
Define Server Actions to handle catalog insertions and stock mutations using Mongoose validation logic.

```
'use server';

import { revalidatePath } from 'next/cache';
import { connectToDatabase } from '@/app/lib/mongodb';
import Product from '@/app/models/Product';

// Server Action: Add New Catalog Product
export async function createProductAction(formData: FormData)
{
  await connectToDatabase();

  const title = formData.get('title') as string;
  const price = parseFloat(formData.get('price') as string);
  const category = formData.get('category') as string;
  const sku = formData.get('sku') as string;
  const stock = parseInt(formData.get('stock') as string,
10);
  const warehouseLocation = formData.get('warehouseLocation')
as string;
  const tagsRaw = formData.get('tags') as string;

  const tags = tagsRaw
    ? tagsRaw.split(',').map((tag) =>
tag.trim()).filter(Boolean)
    : [];
  const slug = title.toLowerCase().replace(/[^a-z0-9]+/g, '-
').replace(/(^-|-$)+/g, '');
```

```
  await Product.create({
    title,
    slug: `${slug}-${Date.now().toString().slice(-4)}`,
    price,
    category,
    tags,
    inventory: {
      sku,
      stock,
      warehouseLocation,
    },
  });

  revalidatePath('/catalog');
}

// Server Action: Delete Product
export async function deleteProductAction(productId: string)
{
  await connectToDatabase();
  await Product.findByIdAndDelete(productId);
  revalidatePath('/catalog');
}
```

Step 5: Construct Catalog Dashboard UI View (app/catalog/page.tsx**)**
Create an asynchronous Server Component that connects to MongoDB, reads catalog documents, and displays operational inventory statuses.

```
import { connectToDatabase } from '@/app/lib/mongodb';
import Product from '@/app/models/Product';
import { createProductAction, deleteProductAction } from
'@/app/actions/catalogActions';

export default async function CatalogPage() {
  await connectToDatabase();

  // Fetch all products sorted by creation time
  const products = await Product.find({}).sort({ createdAt: -
1 }).lean();
```

```
  return (
    <div className="min-h-screen bg-slate-100 text-slate-900
p-6 sm:p-10 font-sans">
      <div className="max-w-4xl mx-auto space-y-8">

        {/* Page Header */}
        <header className="border-b border-slate-200 pb-5">
          <div className="flex items-center gap-2 mb-2">
            <span className="px-2.5 py-0.5 rounded-full text-
xs font-semibold bg-emerald-100 text-emerald-800">
              Experiment 6
            </span>
            <span className="text-xs text-slate-500 font-
mono">
              Next.js 16 + MongoDB & Mongoose
            </span>
          </div>
          <h1 className="text-2xl sm:text-3xl font-bold
tracking-tight text-slate-900">
            E-Commerce Catalog & Inventory Tracking
          </h1>
          <p className="mt-1 text-sm text-slate-600">
            Managing document-oriented catalog models with
embedded schemas and lifecycle middleware.
          </p>
        </header>

        {/* Catalog Entry Form */}
        <section className="bg-white rounded-xl border
border-slate-200 shadow-sm p-6 sm:p-8">
          <h2 className="text-lg font-semibold text-slate-900
mb-1">
            Register Catalog Item
          </h2>
          <p className="text-xs text-slate-500 mb-6">
            Inserts a strictly validated BSON document with
an embedded inventory schema.
          </p>

          <form action={createProductAction}
className="space-y-4">
```

```
            <div className="grid grid-cols-1 sm:grid-cols-3
gap-4">
              <div className="sm:col-span-2">
                <label className="block text-xs font-semibold
text-slate-700 uppercase tracking-wider mb-1">
                  Product Title
                </label>
                <input
                  type="text"
                  name="title"
                  placeholder="e.g. Wireless Noise-Cancelling
Headphones"
                  required
                  className="w-full rounded-lg border border-
slate-300 bg-white px-3 py-2 text-sm text-slate-900
focus:outline-none focus:ring-2 focus:ring-emerald-500"
                />
              </div>

              <div>
                <label className="block text-xs font-semibold
text-slate-700 uppercase tracking-wider mb-1">
                  Category
                </label>
                <select
                  name="category"
                  required
                  className="w-full rounded-lg border border-
slate-300 bg-white px-3 py-2 text-sm text-slate-900
focus:outline-none focus:ring-2 focus:ring-emerald-500"
                >
                  <option
value="Electronics">Electronics</option>
                  <option value="Footwear">Footwear</option>
                  <option value="Apparel">Apparel</option>
                  <option
value="Accessories">Accessories</option>
                  <option value="Home">Home</option>
                </select>
              </div>
            </div>
```

```
            <div className="grid grid-cols-1 sm:grid-cols-3
gap-4">
              <div>
                <label className="block text-xs font-semibold
text-slate-700 uppercase tracking-wider mb-1">
                  Price ($)
                </label>
                <input
                  type="number"
                  step="0.01"
                  name="price"
                  placeholder="199.99"
                  required
                  className="w-full rounded-lg border border-
slate-300 bg-white px-3 py-2 text-sm text-slate-900
focus:outline-none focus:ring-2 focus:ring-emerald-500"
                />
              </div>

              <div>
                <label className="block text-xs font-semibold
text-slate-700 uppercase tracking-wider mb-1">
                  SKU Identifier
                </label>
                <input
                  type="text"
                  name="sku"
                  placeholder="ELEC-WNC-01"
                  required
                  className="w-full rounded-lg border border-
slate-300 bg-white px-3 py-2 text-sm text-slate-900
focus:outline-none focus:ring-2 focus:ring-emerald-500"
                />
              </div>

              <div>
                <label className="block text-xs font-semibold
text-slate-700 uppercase tracking-wider mb-1">
                  Initial Stock
                </label>
```

```
                <input
                  type="number"
                  name="stock"
                  placeholder="25"
                  required
                  className="w-full rounded-lg border border-
slate-300 bg-white px-3 py-2 text-sm text-slate-900
focus:outline-none focus:ring-2 focus:ring-emerald-500"
                />
              </div>
            </div>

            <div className="grid grid-cols-1 sm:grid-cols-2
gap-4">
              <div>
                <label className="block text-xs font-semibold
text-slate-700 uppercase tracking-wider mb-1">
                  Warehouse Location
                </label>
                <input
                  type="text"
                  name="warehouseLocation"
                  placeholder="e.g. WH-East-A3"
                  required
                  className="w-full rounded-lg border border-
slate-300 bg-white px-3 py-2 text-sm text-slate-900
focus:outline-none focus:ring-2 focus:ring-emerald-500"
                />
              </div>

              <div>
                <label className="block text-xs font-semibold
text-slate-700 uppercase tracking-wider mb-1">
                  Tags (Comma Separated)
                </label>
                <input
                  type="text"
                  name="tags"
                  placeholder="audio, bluetooth, wireless"
```

```
                  className="w-full rounded-lg border border-
slate-300 bg-white px-3 py-2 text-sm text-slate-900
focus:outline-none focus:ring-2 focus:ring-emerald-500"
                />
              </div>
            </div>

            <button
              type="submit"
              className="px-6 py-2.5 bg-emerald-600 hover:bg-
emerald-700 text-white text-sm font-semibold rounded-lg
shadow transition duration-150 cursor-pointer"
            >
              Add Catalog Item
            </button>
          </form>
        </section>

        {/* Product Catalog Records Feed */}
        <section className="space-y-4">
          <div className="flex justify-between items-center">
            <h2 className="text-lg font-semibold text-slate-
900">
              Tracked Products ({products.length})
            </h2>
            <span className="text-xs text-slate-500">Mongoose
Collection: <code>products</code></span>
          </div>

          {products.length === 0 ? (
            <div className="bg-white border border-dashed
border-slate-300 rounded-xl p-8 text-center text-slate-500
text-sm">
              No products found in MongoDB. Register an item
using the form above.
            </div>
          ) : (
            <div className="grid grid-cols-1 md:grid-cols-2
gap-4">
              {products.map((p: any) => (
                <div
```

```
                  key={p._id.toString()}
                  className="bg-white border border-slate-200
rounded-xl p-5 shadow-sm flex flex-col justify-between gap-4
hover:border-slate-300 transition"
                >
                  <div className="space-y-2">
                    <div className="flex justify-between
items-start gap-2">
                      <h3 className="text-base font-semibold
text-slate-900 leading-tight">
                        {p.title}
                      </h3>
                      <span className="px-2 py-0.5 rounded
text-xs font-bold bg-slate-100 text-slate-800">
                        ${p.price.toFixed(2)}
                      </span>
                    </div>

                    <p className="text-xs font-mono text-
slate-500">
                      Slug: {p.slug}
                    </p>

                    <div className="flex flex-wrap gap-1.5
pt-1">
                      <span className="px-2 py-0.5 rounded
text-xs font-medium bg-blue-50 text-blue-700 border border-
blue-100">
                        {p.category}
                      </span>
                      {p.inStock ? (
                        <span className="px-2 py-0.5 rounded
text-xs font-medium bg-emerald-50 text-emerald-700 border
border-emerald-100">
                          In Stock ({p.inventory.stock})
                        </span>
                      ) : (
                        <span className="px-2 py-0.5 rounded
text-xs font-medium bg-rose-50 text-rose-700 border border-
rose-100">
                          Out of Stock
```

```
                        </span>
                      )}
                      <span className="px-2 py-0.5 rounded
text-xs font-mono bg-slate-100 text-slate-600">
                        SKU: {p.inventory.sku}
                      </span>
                      <span className="px-2 py-0.5 rounded
text-xs bg-slate-100 text-slate-600">
                        Loc: {p.inventory.warehouseLocation}
                      </span>
                    </div>
                  </div>

                  <div className="pt-2 border-t border-slate-
100 flex justify-between items-center">
                    <div className="flex gap-1">
                      {p.tags.map((t: string, idx: number) =>
(
                        <span key={idx} className="text-
[11px] text-slate-400">
                          #{t}
                        </span>
                      ))}
                    </div>

                    <form
action={deleteProductAction.bind(null, p._id.toString())}>
                      <button
                        type="submit"
                        className="px-2.5 py-1 bg-red-50
hover:bg-red-100 text-red-600 text-xs font-semibold rounded
border border-red-200 transition cursor-pointer"
                      >
                        Remove
                      </button>
                    </form>
                  </div>
                </div>
              ))}
            </div>
          )}
```

```
        </section>

      </div>
    </div>
  );
}
```

Step 6: Validate Catalog Workflow

1. Execute the Next.js development server:

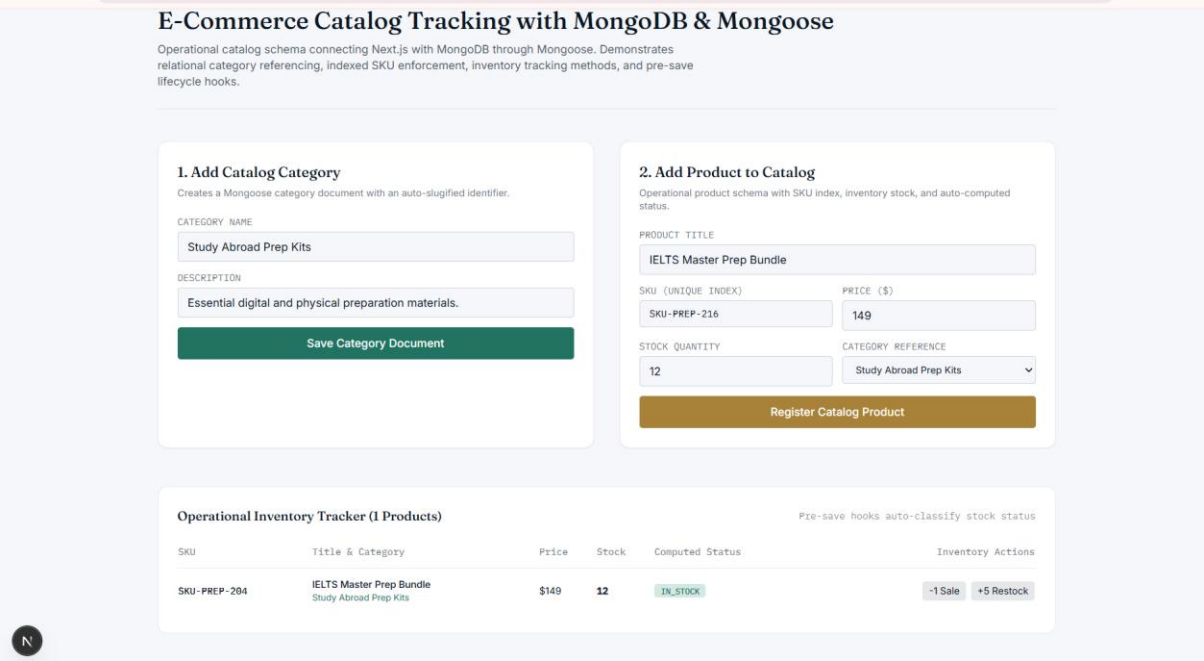
```
npm run dev
```

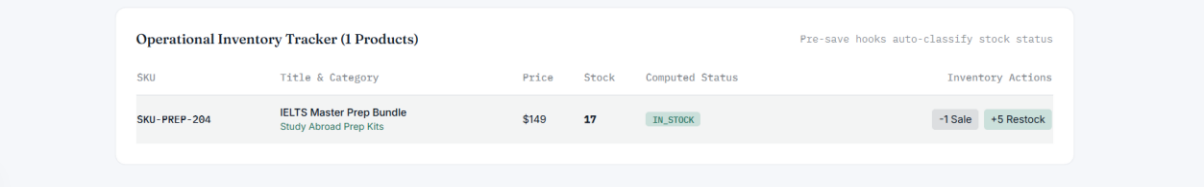
2. Navigate to http://localhost:3000/catalog in your browser.
3. Submit a new catalog item. Inspect the rendered cards to verify that Mongoose computed the slug, validated the category enum, embedded the inventory sub-document, and ran the pre-save hook to evaluate inStock.

OBSERVATION:

Observe how Mongoose enforces document validation rules prior to writing to the MongoDB database. If a non-whitelisted category or negative price is supplied, Mongoose validation halts execution and returns a structured validation error.

Note how embedded schemas (InventorySchema) are stored inline within the parent product document, allowing inventory levels and location coordinates to be read in a single round-trip query without relational joins.





CONCLUSION:

Operational schemas for an e-commerce catalog tracking system were successfully constructed using Next.js and MongoDB via Mongoose. The experiment verified singleton database connection caching, embedded sub-document schema modeling, pre-save lifecycle hooks, and server-side data mutations using React Server Actions.

POST LAB QUESTION:

Q: Why is global connection caching (globalThis.mongooseCache**) mandatory when connecting Mongoose with Next.js App Router applications, and how does it differ from traditional Express.js database connection lifecycles?**

A:

In traditional Express.js architectures, the Node.js server process runs continuously as a long-lived stateful instance where mongoose.connect() is called once at application startup.

In Next.js App Router (especially during development with Hot Module Replacement and in serverless production runtimes), server-side modules are re-evaluated frequently. Without global connection caching, each file modification or server request invokes mongoose.connect(), creating multiple unmanaged database connection sockets.

Caching the connection promise on globalThis guarantees that across hot-reloads and concurrent server operations, Mongoose reuses the existing open connection pool instead of exhausting the database connection limits.